

AI Agent Guide and Master Loop Prompt

Media Reminder - Offline-First Adaptive Media Alarm

Prepared for Mohamed Aslam Abdul

2026-08-05

Contents

0.1 Document control

Field	Value
Document ID	MR-19
Version	1.0
Status	Approved baseline
Last updated	2026-08-05
Product owner	Mohamed Aslam Abdul
Package identifier	com.aslam.mediareminder
Purpose	Provide an authoritative prompt and closed-loop workflow for AI-assisted implementation without scope, security or architecture drift.

Reading rule: This pack specifies a production-oriented Android application, not a promise that third-party devices will behave identically. Where Android or an OEM controls presentation, timing, sound, or permissions, the app provides transparent status and the strongest compliant fallback.

0.2 Document conventions

The words **MUST**, **MUST NOT**, **SHOULD**, **SHOULD NOT**, and **MAY** are normative. A requirement ID is stable once published. Requirement IDs may be retired but **MUST NOT** be reassigned to a different meaning.

Term	Meaning
Reminder	A user-authored instruction that connects content, a schedule, and a presentation profile.
Occurrence	One calculated due instance of a reminder.
Alarm session	The bounded runtime state created when an occurrence is actively alerting.

Term	Meaning
Media asset	An app-owned local video, audio, image, or text item.
Profile	Reusable alert behavior such as Gentle, Standard, or Persistent.
Exact-alarm access	Android special app access that allows exact scheduling where the platform requires it.
Full-screen intent	Android notification mechanism for urgent, time-sensitive activity presentation. It is not a general overlay.
Heads-up notification	System-rendered high-priority notification shown temporarily over the current app when Android permits it.
Source of truth	The authoritative specification or local persisted record for a decision or state.

1 Purpose

This guide is designed for Claude Code, Codex, Cursor or another coding agent working in the repository. The agent operates as an implementation partner, not product owner. It must make small verifiable changes, read the source-of-truth pack, and stop when a decision is missing or evidence fails.

2 Non-negotiable constraints

1. Android-only v1; package `com.aslam.mediareminder`.
2. React Native 0.86.x/New Architecture for normal UI; Kotlin native reliability core.
3. No backend, account, analytics, advertising or production `INTERNET` permission.
4. No overlay permission, broad gallery permission or battery-optimization exemption request.
5. No idle persistent service, time polling or `WorkManager` reminder delivery.
6. One globally earliest ordinary `AlarmManager` event.
7. Full-screen only for locked/non-interactive state and only when platform permits; unlocked uses system notification without FSI.
8. Play/Snooze/Dismiss must work without React Native startup.
9. Media starts only after Play.
10. Room and app-owned files are source of truth; React state is disposable.
11. Imported media is copied, streamed, hashed and transaction-journaled.
12. Backups are logical versioned ZIPs; validate before mutation; never raw database copy.
13. Every background resource is bounded and cleaned up.
14. No medical/emergency guarantee.
15. Any permission, schema, backup or architectural change requires source-of-truth/ADR update.

3 Context loading order

Before coding, read:

1. AGENTS.md;
2. MR-00 Document Map;
3. MR-17 ADRs;
4. the requirement rows in MR-21 for the selected task;
5. the domain specification: MR-06 for alarms, MR-09 for persistence, MR-10 for backup, MR-03/MR-04 for UX;
6. existing code/tests in the exact module;
7. current official docs in MR-22 when platform behavior is involved.

Never rely only on a chat summary when repository documents exist.

4 Work-unit sizing

One loop should implement a coherent vertical slice small enough to review. Good examples:

- occurrence key calculator plus DST tests;
- pending file operation copy/repair state machine;
- exact-alarm capability snapshot and Health row;
- native Dismiss action with idempotency test;
- backup path validator with malicious fixtures;
- one accessible Reminder card component and states.

Bad work units: “build all alarms,” “finish the app,” or a refactor spanning unrelated domains.

5 Master loop prompt

Copy the block below into the coding agent at repository root.

You are the implementation agent for Media Reminder, an offline-first Android adaptive media alarm.

AUTHORITATIVE SOURCES

- Read AGENTS.md and specs/00_Document_Map_and_Executive_Summary.md first.
- Read specs/17_Architecture_Decision_Records.md and the requirement/test rows relevant to this task in specs/21_Requirements_Traceability_and_Acceptance_Catalog.md.
- Read the domain documents referenced by those rows.
- In a conflict, follow the precedence in MR-00. Do not silently choose a new architecture.

HARD CONSTRAINTS

- Android-only v1; React Native UI plus Kotlin native reliability core.
- No backend, account, analytics, ads, Internet permission, overlay permission, broad gallery permission, or default battery-exemption request.
- No persistent idle service, clock polling, repeating alarm, or WorkManager due delivery.
- Schedule one globally earliest one-shot system alarm.
- Full-screen alarm is permitted only when the device is locked or non-interactive and Android permits it. If unlocked, use a normal high-importance notification without a full-screen intent. Android owns heads-up size.
- Native Play, Snooze, Dismiss, timeout and sound stop cannot depend on React Native startup.
- Attached media starts only after Play.
- Room/app-owned files are authoritative. Imported media is streamed into private storage with hash, journal and recovery.
- Backup is a versioned logical ZIP with validation, checksums, staging, conflict plan and rollback. Never import a raw database.
- All file, wake-lock, service, notification and PendingIntent behavior is bounded, explicit and tested.

- Do not add a dependency, permission, exported component, schema field or archive field without explaining its impact and updating required docs/tests.

TASK

Implement exactly this work item: <INSERT ONE ISSUE OR REQUIREMENT SLICE>.

LOOP

1. ORIENT

- Summarize the relevant requirements, invariants and existing code paths.
- List assumptions. Resolve them from repository sources before asking questions.
- Inspect git status and do not overwrite unrelated work.

2. PLAN

- Propose the smallest vertical change.
- Identify files, migrations/contracts, tests, permission/privacy/battery/ally/backup impacts.
- State the observable acceptance result.
- If the plan conflicts with an ADR or lacks a required product decision, STOP and report the exact conflict instead of coding.

3. IMPLEMENT

- Make focused changes following architecture boundaries.
- Use typed domain states/errors, structured concurrency, streaming IO and idempotent native actions.
- Preserve backward compatibility or add explicit migration.
- Keep user-visible strings localized and accessible.
- Do not leave TODO/TBD placeholders in the completed slice.

4. VERIFY FAST

- Run the narrowest relevant formatter, static analysis and tests.
- Fix failures rather than weakening tests.
- Add regression tests for the behavior and failure path.

5. VERIFY SYSTEM

- Run the required broader suites from MR-14/MR-21 for the changed domain.
- For alarm code, include RN-disabled native action and locked/unlocked assertions.
- For file/backup code, include crash/malicious/cancellation cases and bounded-memory behavior.
- For UI, include semantics, large text, dark mode and error/empty/loading states.

6. REVIEW

- Inspect diff for accidental permissions, exported components, logs, paths, secrets, broad refactors and source-of-truth drift.
- Confirm wake locks/services terminate, actions are idempotent and current user data is preserved on failure.
- Confirm no generated/copyrighted private media entered the repository.

7. UPDATE SOURCES

- Update requirement traceability, changelog and specs/ADR only where behavior changed.
- Update fixtures/migration matrices as needed.
- Do not edit generated PDFs manually; regenerate them from Markdown.

8. REPORT

Return:

- implemented requirement IDs;
- concise behavior change;
- files changed;
- tests/commands and results;
- measured evidence when required;
- migrations/compatibility impact;
- privacy, permission, battery and accessibility review;
- residual risks or blockers.

STOP CONDITIONS

- Required decision is absent or conflicts with an Accepted ADR.
- A test exposes data loss, alarm action unavailability, unlocked full-screen takeover, unbounded background work, archive validation bypass, or privacy leak.
- Current official Android behavior differs materially from MR-22.
- The repository has unrelated uncommitted changes that would be overwritten.
- A requested change would require a prohibited permission or false reliability claim.

Never claim completion unless acceptance evidence passes. Never hide a failed test or replace it with a weaker assertion.

6 Agent output contract

Each completed loop reports a table:

Field	Required content
Requirements	IDs implemented/verified
Behavior	User-observable result
Files	Added/changed/deleted
Tests	Exact commands and pass/fail
Evidence	Screenshots, benchmark, device/API where relevant
Data	Schema/archive migration and rollback
Platform	Permission, notification, background impact
Quality	Accessibility/privacy/battery review
Residual	Known risk or None

Do not output unverifiable phrases such as “should work” as final evidence.

7 Task decomposition map

7.1 Foundation sequence

1. repository/manifest guard;
2. domain IDs/time primitives;
3. Room schema/migrations;
4. native Codegen snapshot;
5. media copy journal;
6. library UI;
7. recurrence calculator;
8. scheduler outbox;
9. capability Health;
10. native notification/actions;
11. locked activity/ringing;
12. player transition;
13. boot/time recovery;
14. backup export;
15. import validation/conflict/commit;
16. accessibility/performance/release.

Do not start polished alarm animation before native action state and scheduler idempotency exist.

8 Review heuristics for AI-generated code

Red flags:

- `setInterval`, background timer or repeating alarm;
- `service START_STICKY` without session validation;
- `SYSTEM_ALERT_WINDOW` or launch activity unconditionally from receiver;
- `READ_MEDIA_VIDEO` for a one-file picker use case;
- raw file path passed to React;
- `getBytes()` on `media/ZIP`;
- `fallbackToDestructiveMigration()`;
- mutable implicit `PendingIntent`;
- notification action routed only through JS deep link;
- database copied into ZIP;
- archive extraction before traversal/size validation;
- catch-all exception that continues ringing or deletes temp evidence;
- log statement containing title/path/URI;
- promise of exact 20% heads-up height;
- permission prompt on first launch with no user intent.

9 Prompt for design implementation

For a UI slice, append:

Use MR-03 and MR-04 exactly. Implement all states: normal, loading, empty, error, permission-limited, 200% font, TalkBack semantics, light/dark and compact/medium width. Use tokens rather than ad hoc values. Do not invent a generic SaaS dashboard. Keep copy calm and verb-first.

10 Prompt for adversarial review

After implementation, run a separate review agent with:

Review this diff as a hostile Android reliability, data-integrity and privacy reviewer. Find paths that can: show full screen while unlocked; leave sound/service/wake lock active; duplicate an action; lose media/data during crash; trust a ZIP/path/URI; add network/permission exposure; break backup compatibility; or block TalkBack. Cite files/lines and requirement/ADR. Do not rewrite code until findings are listed.

11 Human responsibility

The human owner reviews permissions, signing, release claims, religious/example content, license provenance and destructive migrations. An AI agent does not approve its own security finding, change ADR status or publish signed binaries without the release checklist.

11.1 Governance

This document is part of the **Media Reminder Source-of-Truth Pack v1.0**. In case of conflict, apply this precedence order: (1) explicit platform safety and permission rules, (2) Architecture Decision Records, (3) Android specification, (4) data and backup specifications, (5) PRD and feature specification, (6) UX and

visual guidance, (7) implementation notes. Record any intentional deviation in the decision log before release.